AI PaaS

GPU Cloud Service Platform — Architecture Document

Distributed P2P | Hyperswarm RPC | OpenAI Compatible | SSE Streaming

Version 1.0 — March 2026

Node.js • Fastify • HyperDB • React

Contents

1. 1. System Overview
2. 2. High-Level Architecture
3. 3. Request Flows
4. 4. Data Layer & HyperDB Schema
5. 5. Authentication & Authorization
6. 6. Rate Limiting
7. 7. Worker Architecture & RPC
8. 8. Load Balancing Strategies
9. 9. Model Runtime Adapter
10. 10. Frontend Architecture
11. 11. Docker & Deployment Topology
12. 12. Scaling & Production Considerations
13. 13. Technology Stack

1. System Overview

AI PaaS is a simplified GPU cloud service platform that provides API key management, model deployment, and OpenAI-compatible inference capabilities. It operates as a **mini AI PaaS** with a fully distributed, peer-to-peer architecture.

Core Capabilities

- API key lifecycle management (create, revoke, rate limits)
- Model deployment to worker nodes with progress streaming
- OpenAI-compatible chat completions (`/v1/chat/completions`)
- Real-time SSE streaming for inference and deployment logs
- Usage metrics and telemetry aggregation

Design Principles

- **No shared singletons** — dependency injection everywhere
- **Swappable strategies** — LB, rate-limiting, model runtime
- **P2P-first** — no central coordinator for worker discovery
- **OpenAI compatibility** — any SDK works by changing baseURL
- **Security-first** — hashed keys, CSRF, encrypted sessions

2. High-Level Architecture

The system is split into three layers: request pipeline, worker compute, and infrastructure.

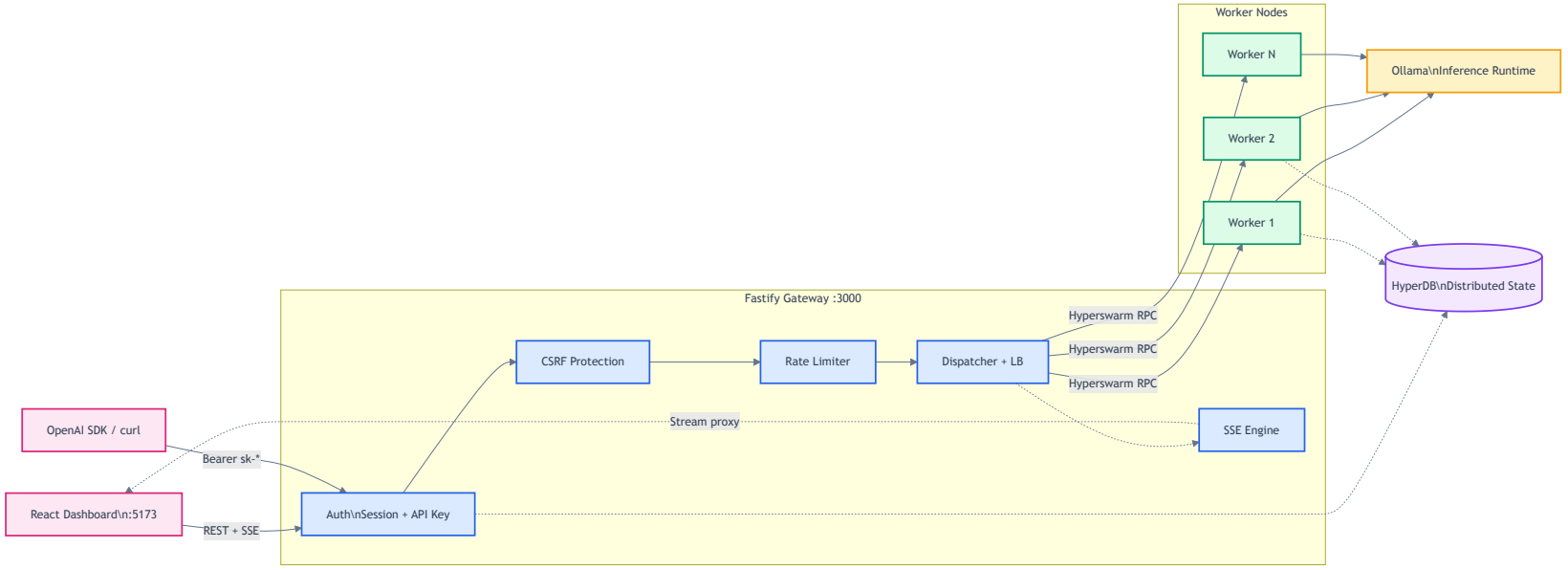


Figure 1a — Request Pipeline (left-to-right: Client → Gateway → Workers → Ollama)

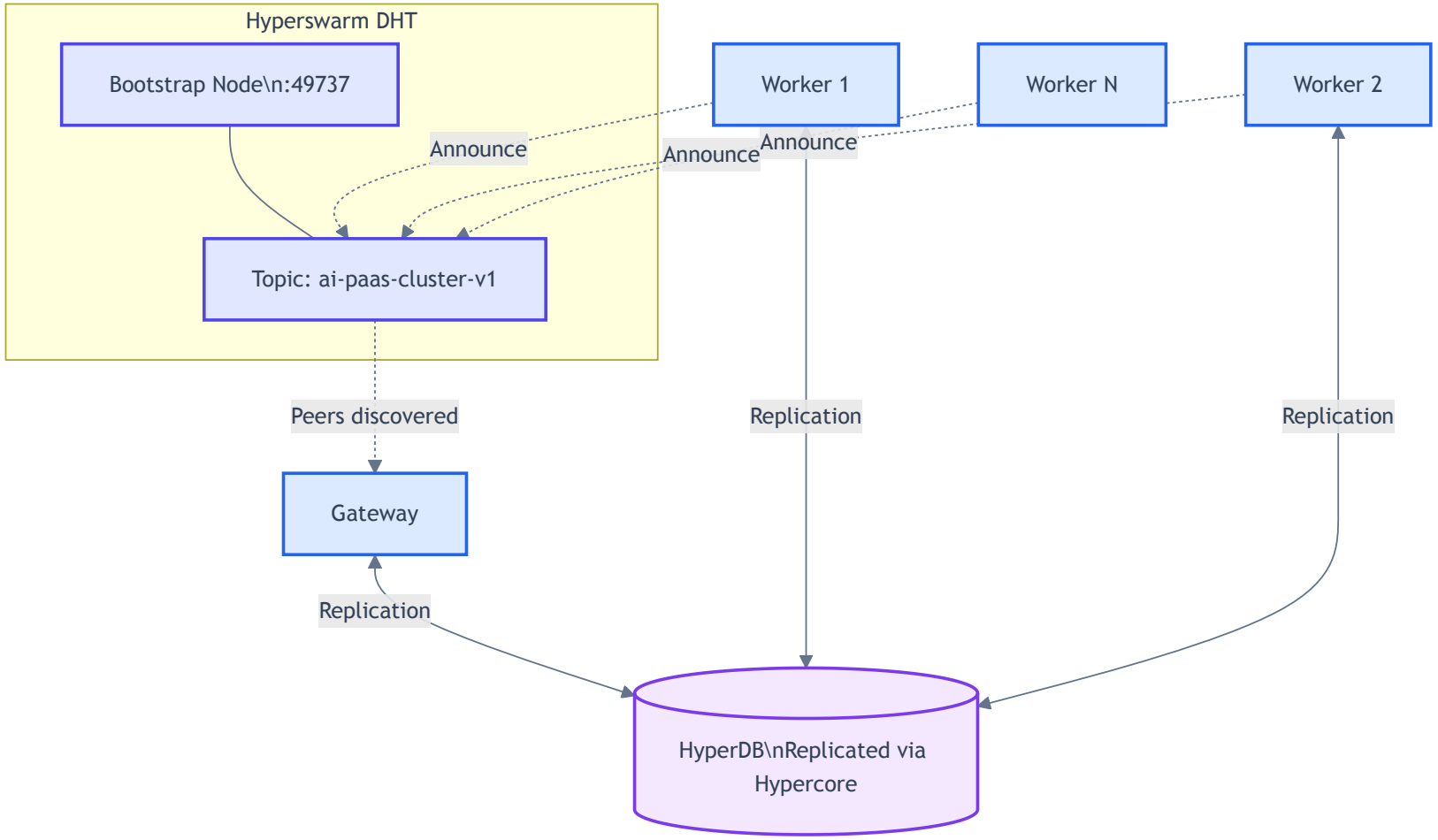


Figure 1b — Infrastructure Layer (Peer Discovery + Data Replication)

Key insight: Workers are discovered automatically via Hyperswarm DHT — no service registry or configuration needed. New workers join by announcing on the cluster topic. HyperDB state is replicated across all nodes via Hypercore protocol.

3. Request Flows

3.1 Inference Flow (Non-Streaming)

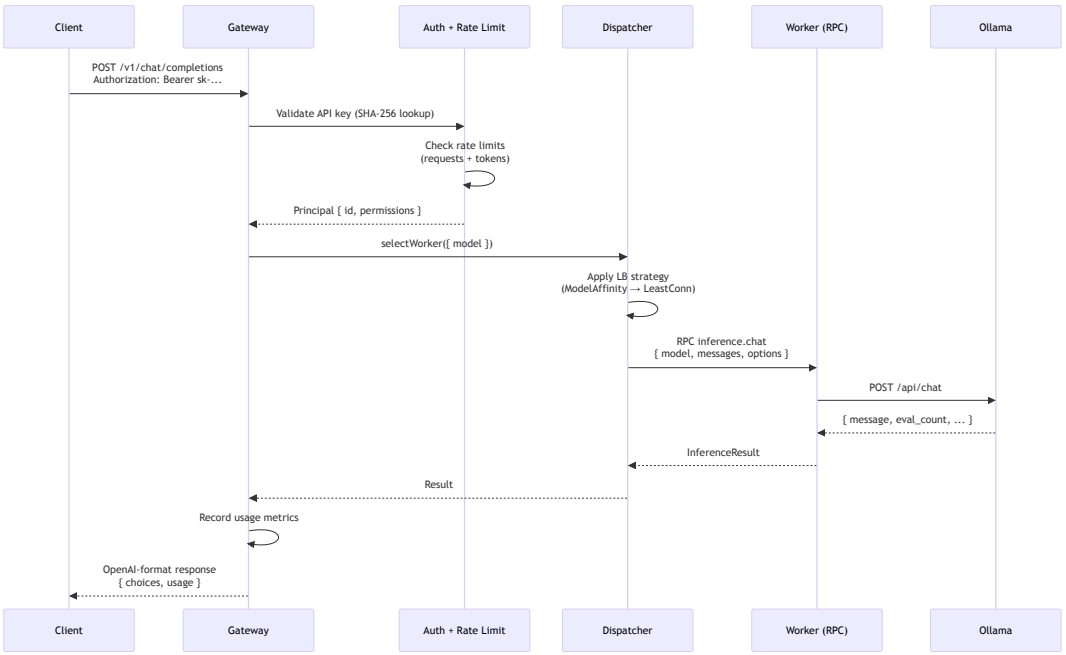


Figure 2 — Inference Request Flow (non-streaming)

3.2 Inference Flow (Streaming)

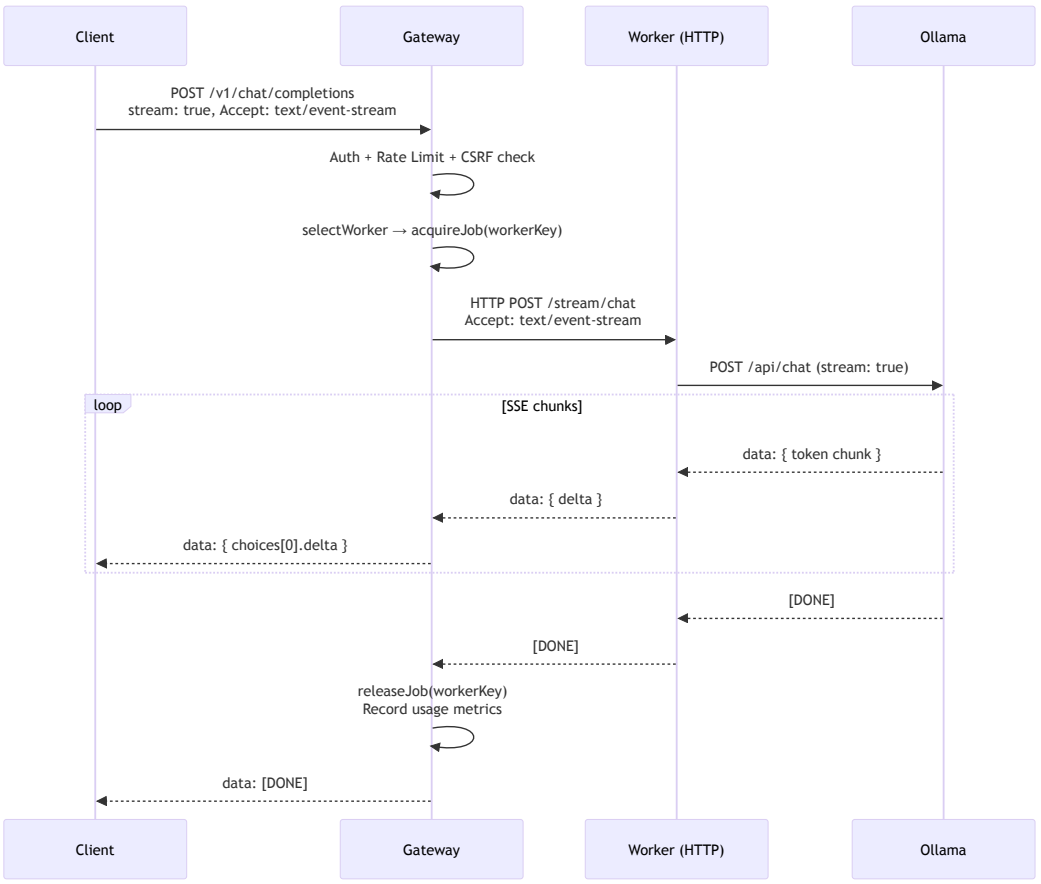


Figure 3 — Streaming Inference via SSE Proxy

Job slot management: The dispatcher acquires a job slot immediately after worker selection (before the HTTP request starts) and releases it in a `releaseOnce()` guard that fires in the generator's `finally` block or a safety-net wrapper.

3.3 Model Deployment Flow

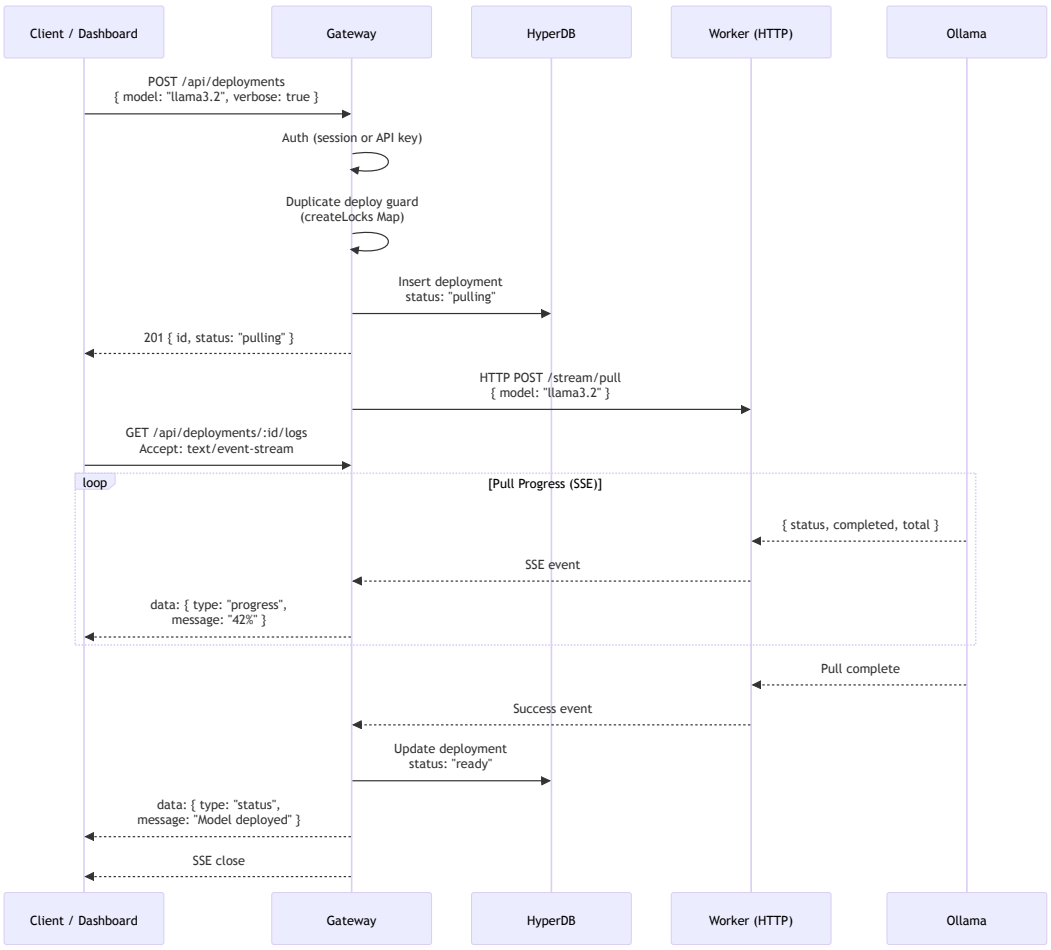


Figure 4 — Model Deployment with Live Streaming Logs

3.4 Authentication Flow

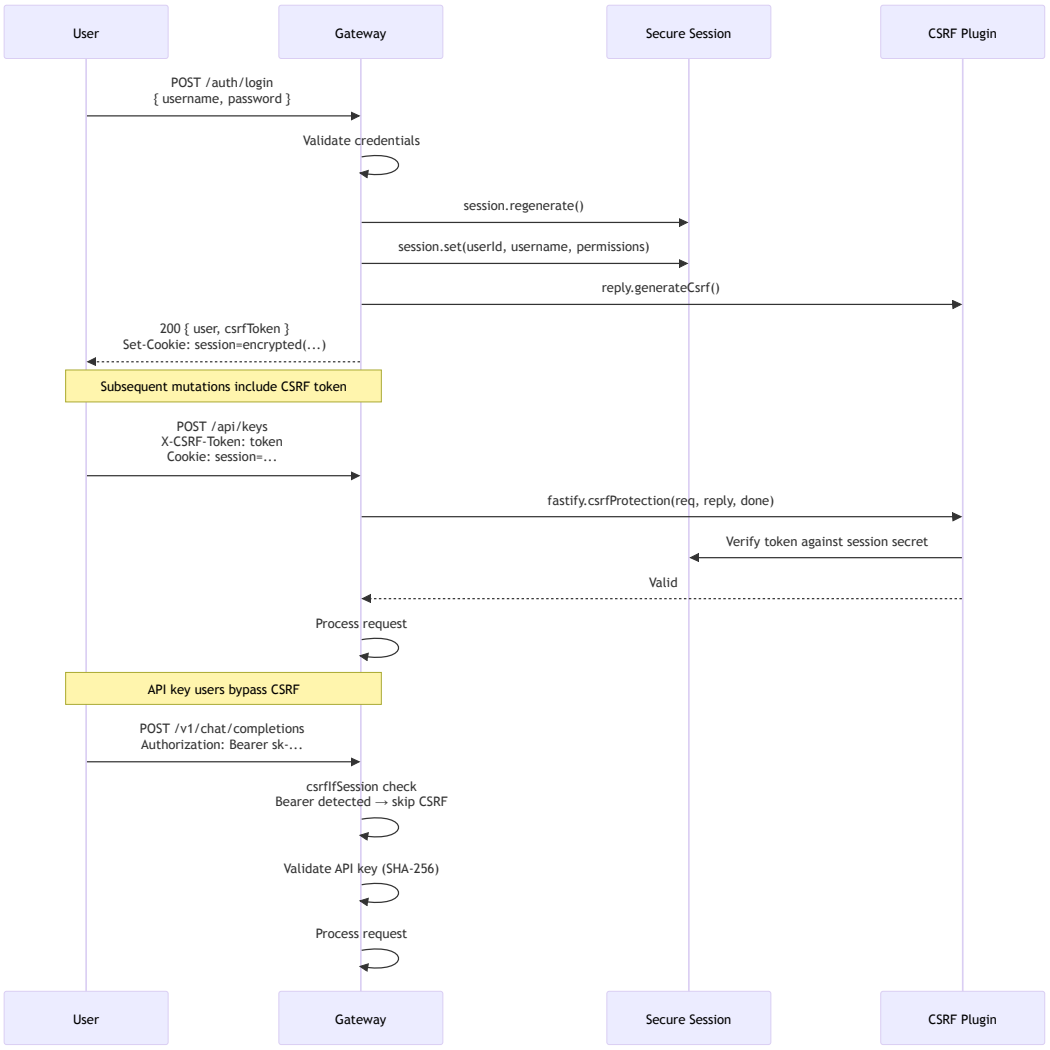


Figure 5 — Dual Authentication Flow (Session + API Key)

4. Data Layer & HyperDB Schema

All state is stored in **HyperDB**, a distributed database built on Hypercore protocol. Data is replicated automatically across all nodes (gateway + workers) via the Hypercore replication protocol. No central database server is needed.

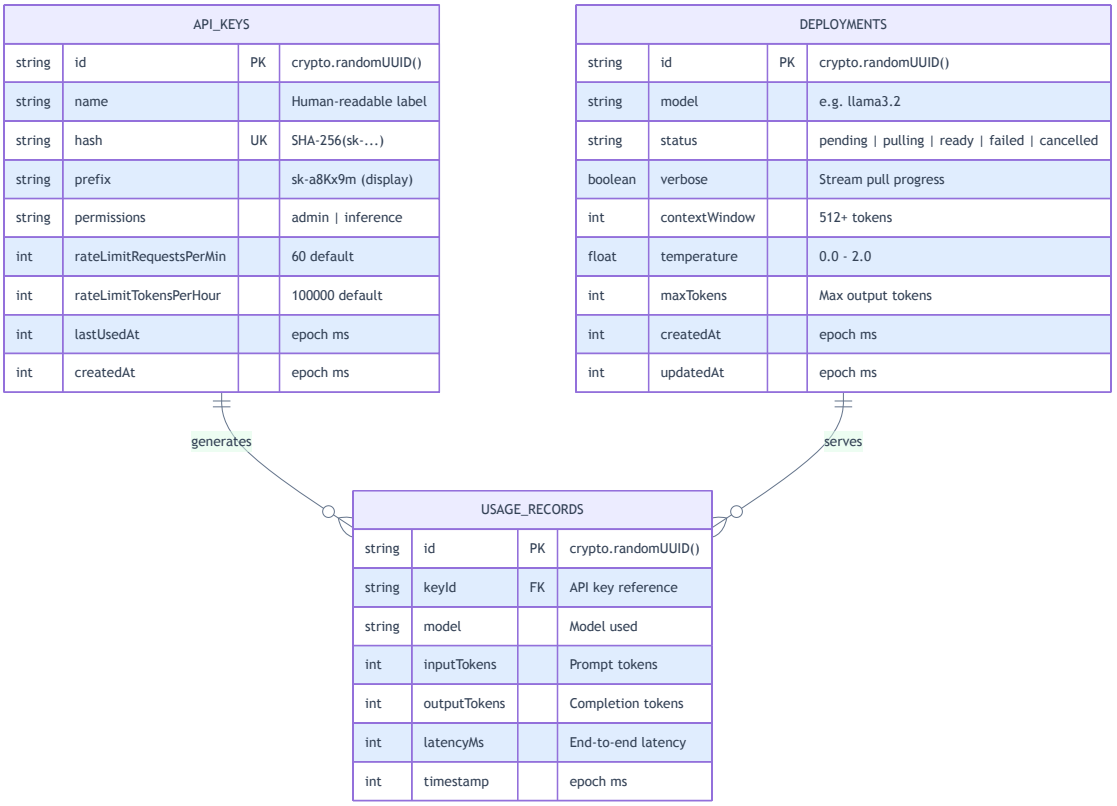


Figure 6 — HyperDB Entity Relationship Diagram

Collections & Indexes

COLLECTION	PRIMARY KEY	INDEXES	PURPOSE
`@aipaas/apiKeys`	`id`	`apiKeys-by-hash` (unique)	O(1) key validation via SHA-256 hash lookup

`@aipaas/deployments`	`id`	`deployments-by-model`	Find deployments for a given model name

`@aipaas/usageRecords`	`id`	`usageRecords-by-keyId`
		`usageRecords-by-model`
		`usageRecords-by-timestamp`
			Per-key usage lookup
			Per-model usage lookup
			Range queries for aggregation (last 24h)

> **Timestamp index optimization:** The `usageRecords-by-timestamp` index uses `IndexEncoder.UINT` for the timestamp field, enabling efficient `gte` / `lte` range queries. Metrics aggregation only reads records from the last 24 hours instead of scanning the entire usage history.

Schema Build Pipeline

HyperDB schemas are defined in `backend/commands/build-schema.ts` and compiled to `spec/hyperdb/` via `HyperDB.toDisk()`. The generated files (`db.json`, `index.js`, `messages.js`) are **never edited manually** — they are rebuilt by running:

```
npx tsx commands/build-schema.ts
```

5. Authentication & Authorization

Dual Authentication Strategy

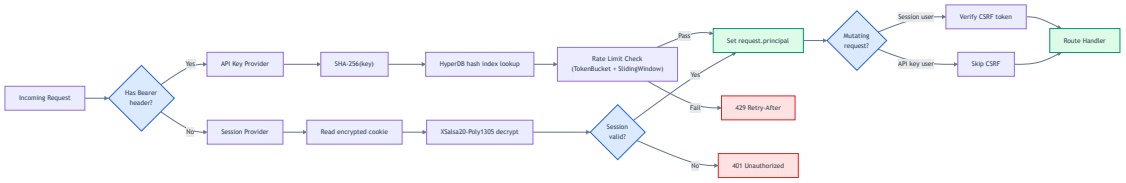


Figure 7 — Authentication Decision Tree

Route Authentication Matrix

ROUTE GROUP	AUTH METHOD	CSRF REQUIRED	RATE LIMITED
`/auth/*`	None (public)	Logout only	No
`/health`	None (public)	No	No
`/api/keys`	Session only	Yes (mutations)	No
`/api/metrics`	Session only	No (read-only)	No
`/api/deployments`	Composite (either)	Session users only	API key users only
`/v1/chat/completions`	Composite (either)	Session users only	API key users only
`/internal/workers`	Session only	No (read-only)	No

API Key Security Model

- **Format:** `sk-` + base62(crypto.randomBytes(32)) — 256-bit entropy
- **Storage:** SHA-256 hash only. Plaintext never stored or logged

- **View-once:** Full key returned exactly once at creation. Cannot be reconstructed
- **Display:** `sk-a8Kx9m••••••••` (prefix only) in dashboard
- **Why SHA-256 (not bcrypt):** Keys are 256-bit random tokens — dictionary attacks are computationally infeasible. Constant-time hash lookup enables O(1) validation

Session Security

- **Encryption:** XSalsa20-Poly1305 (via `@fastify/secure-session`)
- **Session fixation prevention:** `session.regenerate()` on login
- **CSRF:** Token seeded on login via `reply.generateCsrf()`, validated per-route
- **Cookie flags:** `httpOnly`, `secure` (production), `sameSite: strict`

6. Rate Limiting

Composite Architecture

Rate limiting uses a **composite pattern**: multiple strategies are chained, and *all must pass* for a request to proceed. This enables independent limits on request count and token consumption.

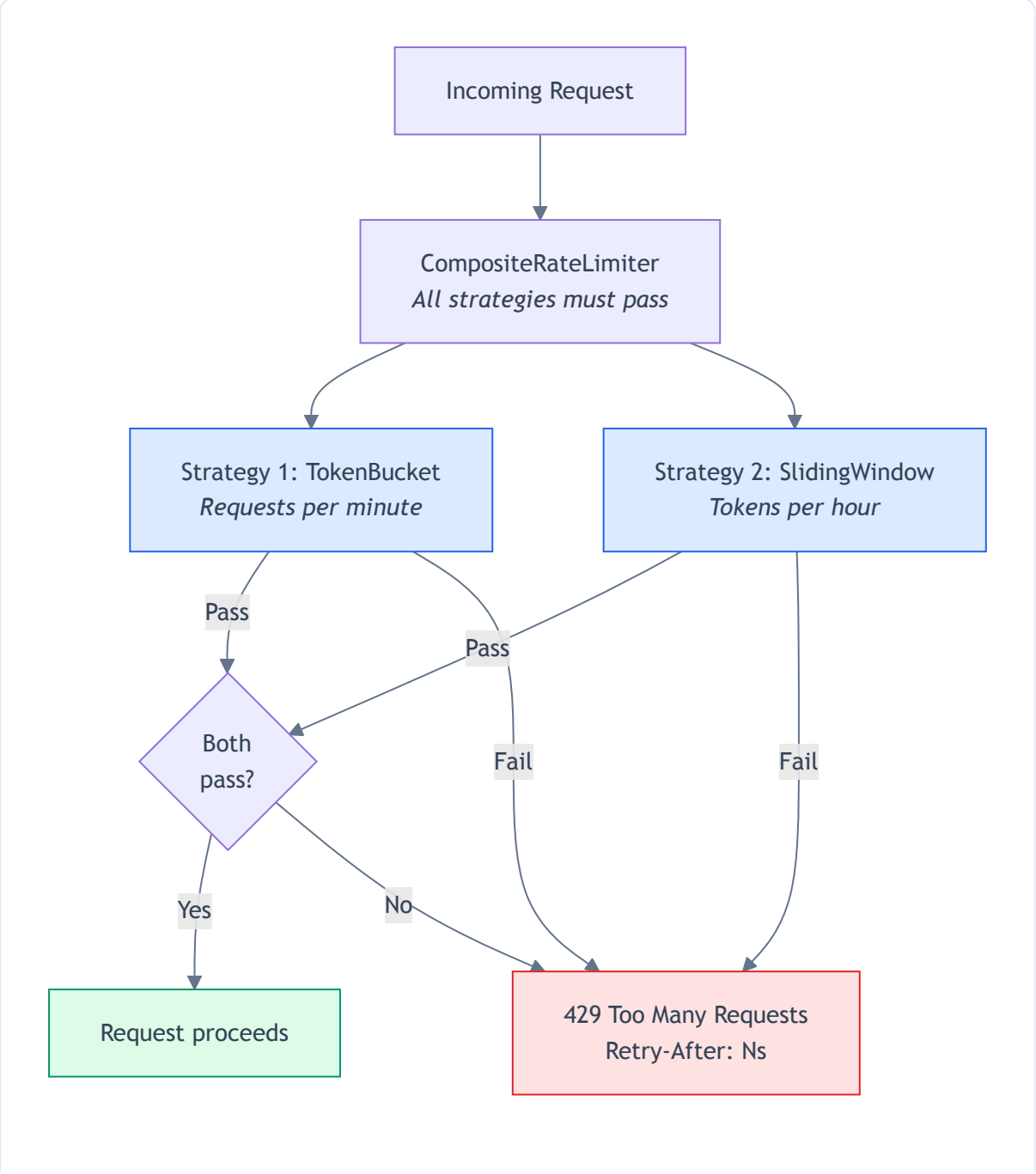


Figure 8 — Composite Rate Limiter

Strategy Implementations

STRATEGY	ALGORITHM	KEY PARAMETERS	USE CASE
FixedWindow	Time-bucketed counters; `Math.floor(now / windowMs)`	Window size, max requests	Simple per-minute caps
SlidingWindow	Timestamp log per identifier; filters entries outside window	Window duration, limit	Token consumption (per hour)
TokenBucket	Refillable bucket; tokens += (elapsed / refillInterval) * refillRate	Capacity, refill rate, interval	Request rate (per minute, bursty)

Per-Key Configuration

Each API key stores its own rate limit thresholds:

- `rateLimitRequestsPerMin` — Requests per minute (default: 60)
- `rateLimitTokensPerHour` — Total tokens per hour (default: 100,000)

The `ApiKeyAuthProvider` creates a composite limiter per key on first use and caches up to 1,000 limiters. Stale entries are evicted LRU-style.

7. Worker Architecture & RPC

Worker Registration & Discovery

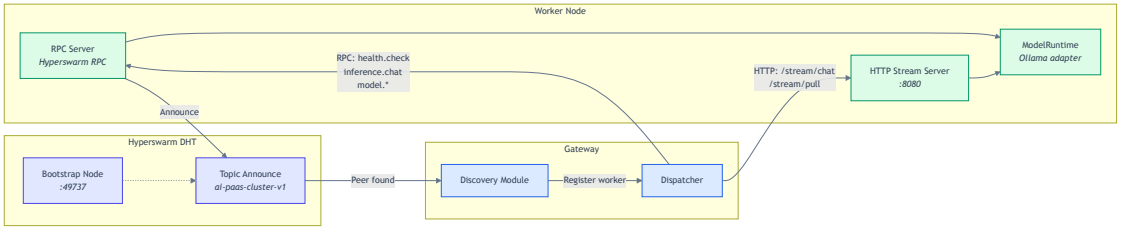


Figure 9 — Worker Discovery & Communication

RPC Methods

METHOD	TRANSPORT	DIRECTION	PURPOSE
`health.check`	Hyperswarm RPC	Gateway → Worker	Heartbeat (30s interval, 10s timeout, 3 fails = unhealthy)
`model.list`	Hyperswarm RPC	Gateway → Worker	List loaded models on worker
`model.pull`	Hyperswarm RPC	Gateway → Worker	Trigger model download (non-verbose)
`model.delete`	Hyperswarm RPC	Gateway → Worker	Remove model from worker
`model.show`	Hyperswarm RPC	Gateway → Worker	Get model metadata (size, parameters, family)
`inference.chat`	Hyperswarm RPC	Gateway → Worker	Non-streaming inference
`/stream/chat`	HTTP SSE	Gateway → Worker	Streaming inference (token-by-token)

`/stream/pull`	HTTP SSE	Gateway → Worker	Streaming model pull progress

> **Dual transport:** RPC (Hyperswarm) handles request-response patterns. HTTP SSE handles streaming — the gateway proxies SSE frames from worker to client, managing job slots with `acquireJob` / `releaseJob` guards.

Health Check Protocol

- **Interval:** 30 seconds per worker
- **Timeout:** 10 seconds per check
- **Failure threshold:** 3 consecutive failures marks worker as unhealthy
- **Recovery:** Single successful check restores healthy status
- **Unhealthy workers:** Excluded from load balancer selection

8. Load Balancing Strategies

Load balancing uses the **Strategy pattern**. All strategies implement the same `LBStrategy` interface and are swappable at configuration time.

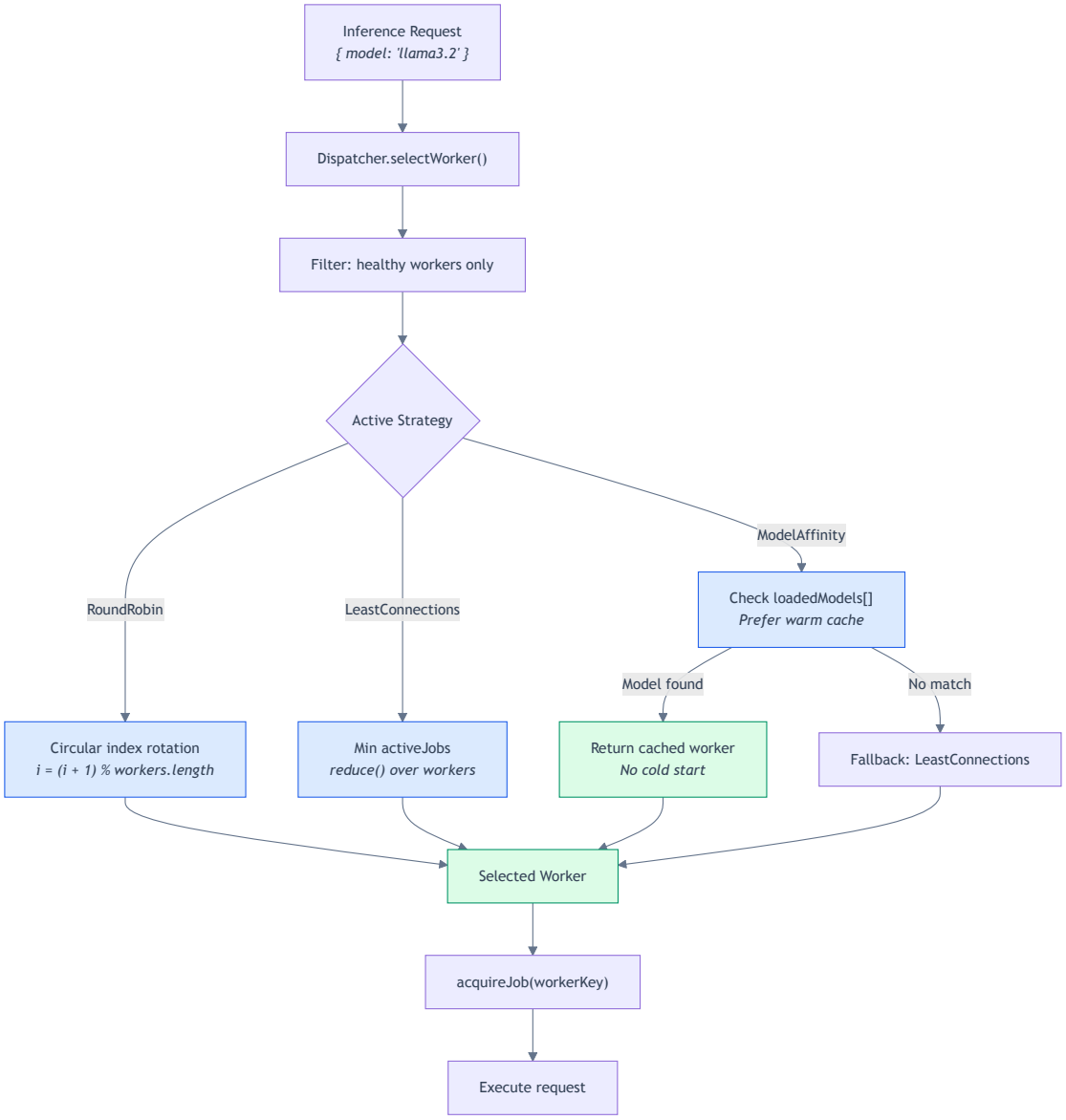


Figure 10 — Load Balancing Strategy Selection

STRATEGY	BEST FOR	TRADE-OFF

RoundRobin	Homogeneous workers, even load	Ignores actual load; may hit busy workers
LeastConnections	Heterogeneous load patterns	Doesn't consider model locality
ModelAffinity	Avoiding cold starts on large models	Can cause hot-spots if one worker has popular model

> **Production note:** A fourth strategy, **WeightedCapacity** (GPU memory + active jobs + queue depth), is documented as a production upgrade. It would require workers to report GPU utilization in health checks.

9. Model Runtime Adapter

Workers use a **ModelRuntime adapter interface** to decouple from any specific inference backend. All model operations go through this abstraction layer, making the runtime swappable without changing worker logic.

Figure 11 — Model Runtime Adapter Pattern (Strategy)

Runtime Selection

```
MODEL_RUNTIME=ollama    # Environment variable selects active runtime
                         # Factory instantiates correct adapter at worker startup
```

Production Alternatives

RUNTIME	ADVANTAGE	TRADE-OFF
Ollama (current)	Simple setup, pull progress streaming, mature API	No continuous batching, single-request inference
vLLM	Continuous batching, PagedAttention, high throughput	More complex deployment, GPU-only
TGI	HuggingFace ecosystem, production-grade	Less flexible model format support

Docker Model Runner	Native Docker integration, fewer moving parts	Newer, less community tooling

10. Frontend Architecture

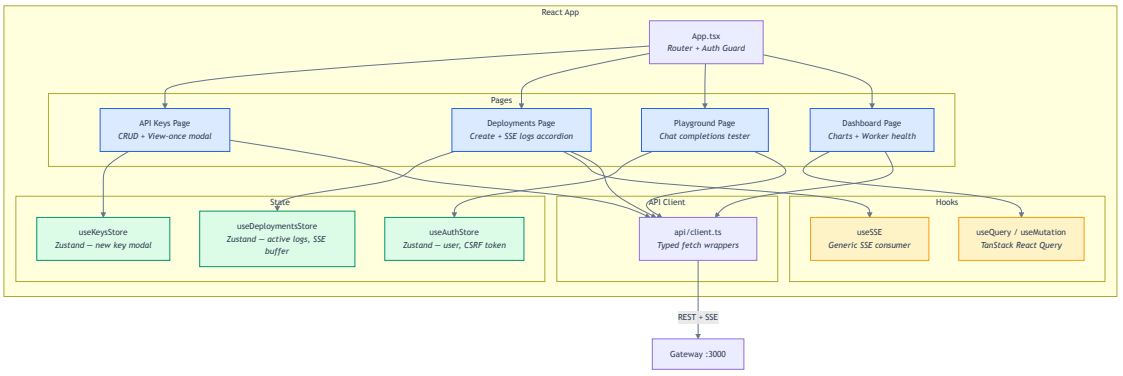


Figure 12 — Frontend Component Architecture

Key Patterns

- **State management:** Zustand for local UI state (modals, SSE buffers); TanStack Query for server state (cache, refetch, optimistic updates)
- **SSE consumption:** Custom `useSSE` hook parses `data: {json}` lines with AbortController cleanup
- **CSRF:** Token stored in auth store, attached to non-GET requests via `X-CSRF-Token` header
- **Auth:** Session cookie with `credentials: 'include'`. No API keys stored client-side
- **Charts:** Recharts for usage visualization (token consumption, latency, requests over time)

11. Docker & Deployment Topology

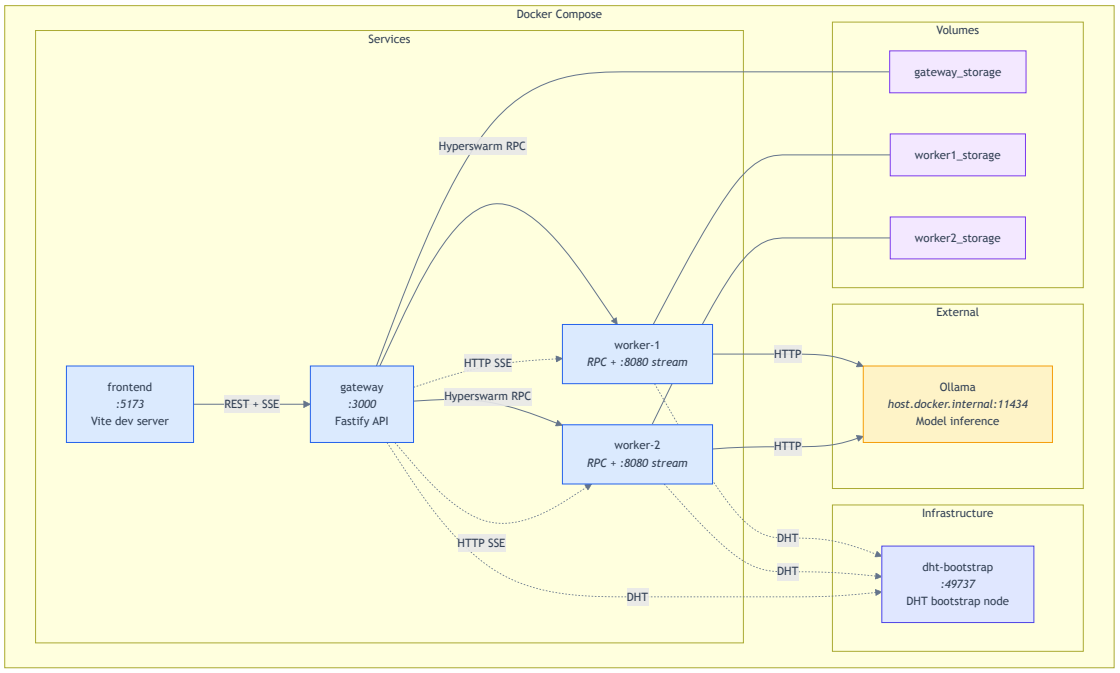


Figure 13 — Docker Compose Service Topology

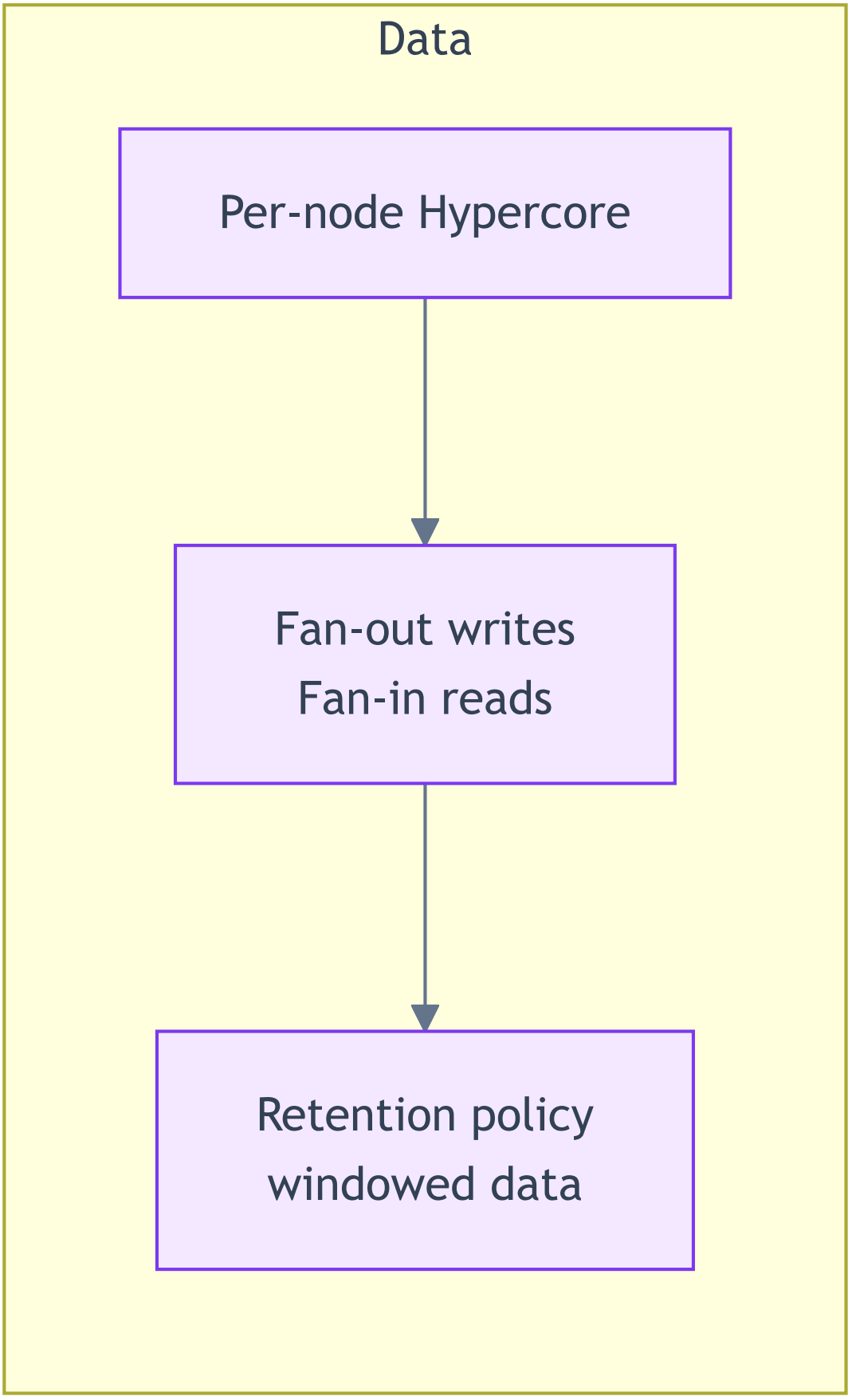
Key Configuration

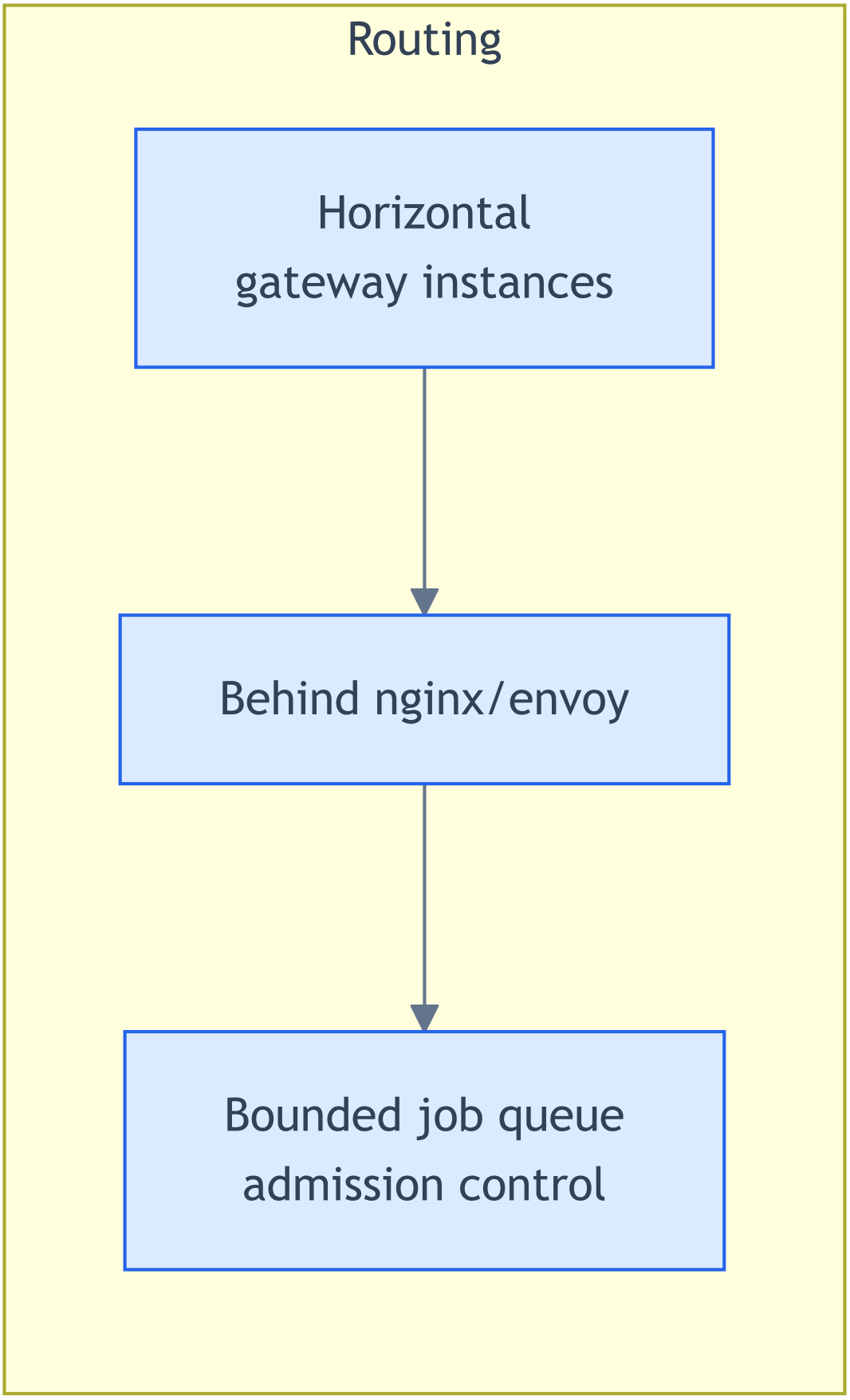
SETTING	VALUE	NOTES
Base image	`node:22-slim`	Not Alpine — `udx-native` lacks linux-arm64-musl prebuild
Hot reload	Source bind mounts	`./backend/src:/app/src` etc.
node_modules	Anonymous volumes	Prevents host override; requires `--renew-anon-volumes` on dep change
DHT networking	`firewalled: false`	Required for Docker bridge networking

Ollama access	`host.docker.internal`	Workers reach host-installed Ollama

12. Scaling & Production Considerations

Scaling Axes


Data
Per-node Hypercore
Fan-out writes
Fan-in reads
Retention policy
windowed data


Routing
Horizontal gateway instances
Behind nginx/envoy
Bounded job queue admission control

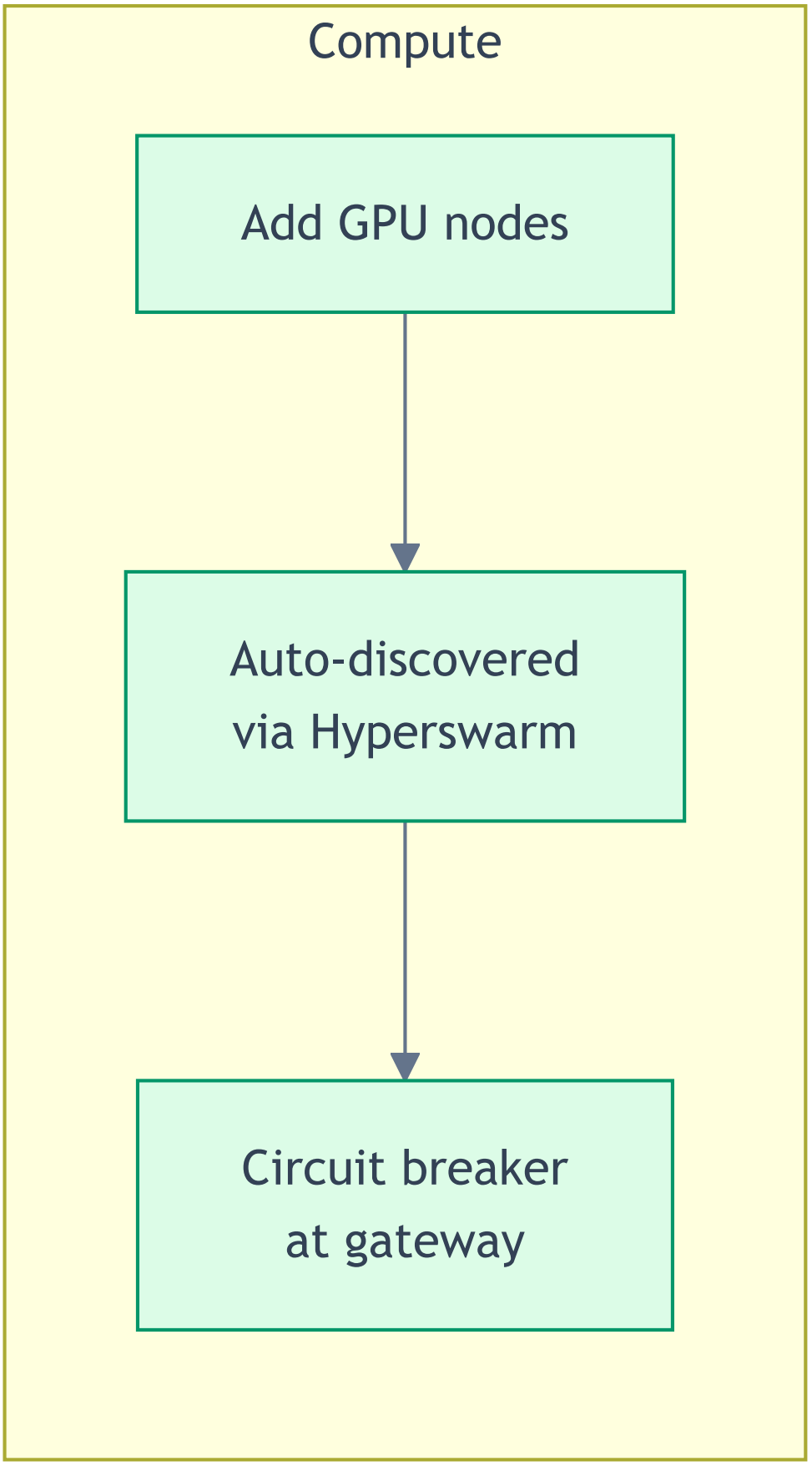


Figure 14 — Three Scaling Axes

Production Upgrades

CURRENT	PRODUCTION ALTERNATIVE	WHY
Ollama	vLLM / TGI	Continuous batching, PagedAttention, higher throughput
HyperDB	PostgreSQL + Redis	Strong consistency for billing; Redis for rate-limit hot path
Single gateway	Gateway cluster + envoy	Health-check routing, TLS termination, connection pooling
In-memory rate limits	Redis-backed limits	Shared state across gateway instances
EnvUserService	Database-backed user store	Multi-user, roles, team management

Failure Modes & Mitigations

FAILURE	DETECTION	MITIGATION
Worker crash	Health check timeout (3 failures)	Mark unhealthy; exclude from LB; 503 + Retry-After if no workers
Ollama down	`isHealthy()` fetch fails	Worker reports unhealthy; gateway circuits open
Gateway crash	Docker restart policy	`recoverStuckDeployments()` on startup
Stream disconnect	`reply.sse.isConnected` check	`releaseOnce()` guard releases job slot
DHT partition	No new peer discoveries	Existing connections maintained; bootstrap node reconnect

13. Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Runtime	Node.js 22	Server runtime (task requirement)
HTTP Framework	Fastify v5	High-performance, plugin ecosystem, schema validation
Worker Comms	@hyperswarm/rpc	P2P RPC over DHT — no central coordinator
Peer Discovery	Hyperswarm	Topic-based peer finding via DHT
Data Store	HyperDB + Hypercore	Distributed, replicated state across all nodes
Model Runtime	Ollama (swappable)	Local LLM inference via adapter interface
SSE	@fastify/sse	Server-Sent Events for streaming
Auth (session)	@fastify/secure-session	XSalsa20-Poly1305 encrypted cookie sessions
CSRF	@fastify/csrf-protection	Token-based CSRF with session backend
API Docs	@fastify/swagger + swagger-ui	Auto-generated OpenAPI spec
Schema Validation	TypeBox (@sinclair/typebox)	Runtime JSON Schema + TypeScript types
Frontend	React + Vite	Dashboard SPA with HMR
Client State	Zustand	Lightweight, no boilerplate local state
Server State	TanStack React Query	Cache, refetch, SSE integration

Charts	Recharts	Usage/metrics visualization
Linter/Formatter	Biome	Single tool — no eslint, no prettier
Package Manager	pnpm	Strict, fast, disk-efficient
Containers	Docker Compose	Local dev with hot reload

AI PaaS Architecture Document — Version 1.0

March 2026